Docker + .NET 8 Laboratory

Goal

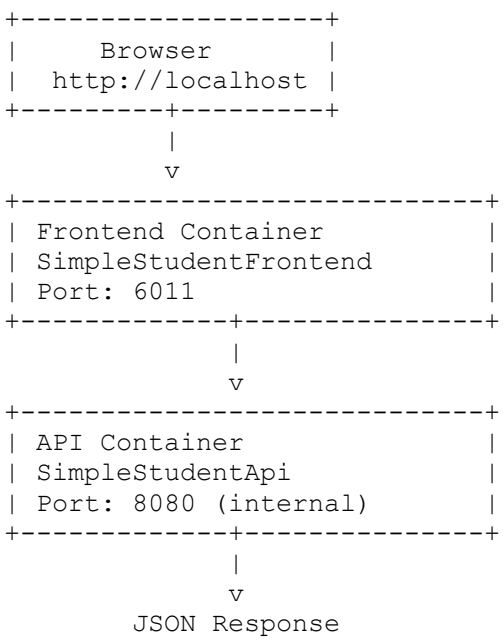
The goal of this lab is to learn how to:

- run a .NET application locally
- publish a .NET application
- build a Docker image
- run an application inside a Docker container
- run **multiple containers** using Docker Compose
- understand **communication between containers**

At the end of this lab you will have:

```
Browser
   |
   v
Frontend (.NET Razor Pages container)
   |
   v
Backend (.NET Minimal API container)
```

System Architecture



Important concept:

```
Frontend does NOT call localhost
Frontend calls container name
```

Example:

```
http://simplestudentapi:8080/api/info
```

Project Structure

```
StudentDockerLab
│
├─ SimpleStudentApi
│    ├─ Program.cs
│    ├─ SimpleStudentApi.csproj
│    └─ Dockerfile
│
├─ SimpleStudentFrontend
│    ├─ Pages
│    ├─ Program.cs
│    ├─ SimpleStudentFrontend.csproj
│    └─ Dockerfile
│
├─ docker-compose.yml
└─ README.md
```

Prerequisites

Install:

- .NET 8 SDK

  https://dotnet.microsoft.com/download

- Docker Desktop

  https://www.docker.com/products/docker-desktop

Verify installation:

```
dotnet --version
docker --version
docker compose version
```

Step 1 — Run API locally

Navigate to the API project:

```
cd SimpleStudentApi
dotnet run
```

Open browser:

```
http://localhost:5071/api/info
```

Expected result:

```
{
  "service": "SimpleStudentApi",
  "version": "1.0",
  "status": "ok"
}
```

Step 2 — Publish API

Create release build:

```
dotnet publish -c Release -o ./publish
```

Check output:

```
ls publish
```

Important files:

```
SimpleStudentApi.dll
SimpleStudentApi.runtimeconfig.json
SimpleStudentApi.deps.json
```

Step 3 — Build Docker image

Inside the API folder:

```
docker build -t simplestudentapi .
```

Check images:

```
docker images
```

Step 4 — Run API container

```
docker run -d \
--name simplestudentapi \
-p 6001:8080 \
-e ASPNETCORE_HTTP_PORTS=8080 \
simplestudentapi
```

Test:

```
http://localhost:6001/api/info
```

or

```
curl http://localhost:6001/api/info
```

Step 5 — Run full system with Docker Compose

From project root:

```
docker compose up --build
```

This will start:

Container	Port	Purpose
simplestudentapi	6001	Backend API
simplestudentfrontend	6011	Frontend web

Step 6 — Test full system

Frontend:

```
http://localhost:6011
```

Frontend will call API container and display data.

API direct access:

```
http://localhost:6001/api/info
```

Container Communication

Important rule:

Containers inside Docker network communicate using **service names**.

Example from docker-compose.yml:

```
services:
  simplestudentapi:
  simplestudentfrontend:
```

Frontend calls:

```
http://simplestudentapi:8080/api/info
```

NOT:

```
http://localhost:6001/api/info
```

Because **localhost inside container = container itself**.

Useful Docker Commands

Show running containers:

```
docker ps
```

Show logs:

```
docker logs simplestudentapi
```

Stop containers:

```
docker compose down
```

Rebuild containers:

```
docker compose up --build
```

Remove everything:

```
docker compose down --volumes
```

Common Problems

Port already in use

Example error:

```
bind: address already in use
```

Solution:

Change port in docker-compose.yml.

Example:

```
6002:8080
```

Browser shows ERR_UNSAFE_PORT

Some ports (example 6000) are blocked by browsers.

Use:

```
6001
6002
7000
8081
```

Container cannot reach API

If frontend cannot call API, check:

```
docker logs simplestudentfrontend
```

Make sure API URL uses **service name**, not localhost.

Correct:

```
http://simplestudentapi:8080/api/info
```

Wrong:

```
http://localhost:6001/api/info
```

Learning Outcomes

After completing this lab students should understand:

- difference between **build and publish**
- difference between **image and container**
- difference between **host port and container port**
- container networking
- service-to-service communication
- Docker Compose basics

Optional Extensions

Students can extend this lab by:

1️⃣ adding new API endpoints

2️⃣ adding Swagger

3️⃣ adding SQL Server container

4️⃣ adding environment variables

5️⃣ creating multiple API replicas

Notes

Key concepts during the lab:

1️⃣ Publish vs Build

```
dotnet build → development
dotnet publish → deployment
```

2️⃣ Container vs Image

```
Image = blueprint
Container = running instance
```

3️⃣ Port Mapping

```
HOST_PORT : CONTAINER_PORT
```

Example:

```
6011 : 8080
```

4️⃣ Docker Network

Containers communicate via **service names**.